

Book Market MVC + DB 연동 웹 애플리케이션 상세 주석 소스코드집

모든 Java 및 JSP 프레젠테이션 레이어 소스코드 전면 주석화 및 가이드라인

[프로젝트 구성 아키텍처 개요]

- **Controller 레이어:** **BookMarket3Controller.java** (모든 브라우저 요청 제어 및 흐름 제어)
- **Model 레이어:** **Book.java** (도서 엔티티 DTO), **BookDAO.java** (H2 DB JDBC 연결 가동 및 CRUD)
- **View 레이어 (JSP):** 메인 웰컴 레이아웃, 공통 네비게이션/푸터, 회원 전용 보안 뷰, 도서 조회/등록 인풋 폼 구조
- **Config 레이어:** 의존성 빌드 가이드 **pom.xml**, 톰캣 서버 로컬 이미지 가상 매핑 설정을 담은 **server.xml**

1. Java Controller 레이어 소스코드

■ BookMarket3Controller.java

```
package ex11;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.MultipartConfig;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.ServletConfig;
import javax.servlet.http.HttpSession;
import javax.servlet.http.Part;
import org.apache.commons.beanutils.BeanUtils;
import java.lang.reflect.Method;
import java.sql.SQLException;
import java.util.List;
import java.util.StringTokenizer;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletContext;

/**
 * [WebServlet 어노테이션]
 * 브라우저에서 "/bm3control" 주소로 요청이 들어오면 이 서블릿이 실행되도록 매핑 */
@WebServlet("/bm3control")

/**
 * [MultipartConfig 어노테이션]
 * 파일 업로드(Multipart) 기능을 지원하도록 서블릿을 설정
 * - maxFileSize: 업로드할 파일의 최대 크기를 지정 (여기서는 2MB 제한) * -
location: 업로드된 임시 파일이 저장될 물리적인 서버 경로를 지정
 */
```

```

@MultipartConfig(maxFileSize=1024 * 1024 * 2, location="c:/Temp/img")
public class BookMarket3Controller extends HttpServlet {
    private static final long serialVersionUID = 1L;

    // 데이터베이스 자원에 접근하기 위한 Data Access Object 객체 변수 선언
    BookDAO dao;

    // 서블릿 전역 환경 정보 및 로그 기록을 담당하는 Context 객체 변수 선언
    private ServletContext ctx;

    /**
     * [init 메서드]
     * 서블릿이 톰캣 서버에 의해 최초로 로드될 때 '딱 한 번' 실행되는 초기화 메서드
    */
    @Override
    public void init(ServletConfig config) throws ServletException {
        super.init(config);
        dao = new BookDAO(); // 데이터베이스 연동을 위한 DAO 객체 생성
        ctx = getServletContext(); // 웹 어플리케이션 전역 Context 정보 획득
    }

    /**
     * [service 메서드]
     * GET, POST 방식에 상관없이 모든 HTTP 요청이 거쳐 가는 핵심 컨트롤러 게이트웨이
    */
    @Override
    protected void service(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // 한글 깨짐 방지를 위해 요청 데이터의 인코딩을 UTF-8로 설정
        request.setCharacterEncoding("UTF-8");

        // URL 파라미터에서 어떤 기능을 요청했는지 구별하기 위한 action 값을 가져옴 (예: ?action=login)
        String action = request.getParameter("action");
        String view = ""; // 최종적으로 이동할 JSP 파일명을 저장할 변수

        // 만약 action 값이 없다면 기본 페이지(welcome)로 강제 포워딩 처리
        if(action == null) {
            getServletContext().getRequestDispatcher("/bm3control?
action=welcome").forward(request, response);
        } else {
            // 들어온 action 명령값에 따라 분기하여 적절한 비즈니스 로직(메서드)을 실행
            switch(action) {
                case "welcome":
                    view = welcome(request, response);
                    break;
                case "login":
                    view = login(request, response);
                    break;
                case "welcomeUser":
                    view = welcomeUser(request, response);

```

```

        break;
    case "productInfoList":
        view = productInfoList(request, response);
        break;
    case "productInfo":
        view = productInfo(request, response);
        break;
    case "addProduct":
        view = addProduct(request, response);
        break;
    case "processLogin":
        view = processLogin(request, response);
        break;
    case "processLogout":
        view = processLogout(request, response);
        break;
    case "processAddProduct":
        view = processAddProduct(request, response);
        break;
    }

    // 핵심 분기: 결정된 view(JSP 파일명) 경로로 요청(request)과 응답(response) 객체를 넘겨 화면을 띄움
    // 모든 결과 화면(JSP)은 웹 프로젝트 내 '/ex11/' 폴더 밑에 위치
    getServletContext().getRequestDispatcher("/ex11/" + view).forward(request,
response);
    }
}

// [웰컴 페이지 핸들러 메서드] 비회원 기본 메인 화면 파일명을 매핑 반환
private String welcome(HttpServletRequest request, HttpServletResponse response) {
    return "welcome.jsp";
}

// [로그인 페이지 핸들러 메서드] 사용자 아이디 입력 폼 뷰 파일명을 매핑 반환
private String login(HttpServletRequest request, HttpServletResponse response) {
    return "login.jsp";
}

// [회원 환영 페이지 핸들러 메서드] 인증 성공 후 접속될 세션 연동 메인 화면 파일명을 반환
private String welcomeUser(HttpServletRequest request, HttpServletResponse response) {
    return "welcomeUser.jsp";
}

/**
 * [도서 목록 조회 메서드]
 * DB에 저장된 전체 도서 리스트를 긁어와 화면에 뿌릴 준비를 함
 */
private String productInfoList(HttpServletRequest request, HttpServletResponse response) {
    // DAO를 통해 데이터베이스(H2)에서 전체 도서 목록을 꺼내와서 List 객체에 담고,
    // 이를 'books'라는 이름으로 request 영역에 보관 (JSP에서 사용 가능하도록)

```

```

        request.setAttribute("books", dao.findAll());
        return "productInfoList.jsp"; // 도서 목록 화면으로 이동
    }

    /**
     * [도서 상세 정보 조회 메서드]
     * 사용자가 클릭한 특정 도서의 고유 ID를 받아와 상세 정보를 조회
    */
    private String productInfo(HttpServletRequest request, HttpServletResponse response) {
        // 수신 인코딩을 한글 깨짐 방지용으로 추가 재설정
        try { request.setCharacterEncoding("UTF-8"); } catch(Exception e){}

        // URL에 묻어온 도서 id(?id=ISBN1234) 파라미터를 받아와 DB에서 단 건의 도서 정보를 조회한 후,
        // 'b'라는 이름으로 request 영역에 세팅
        request.setAttribute("b", dao.find(request.getParameter("id")));
        return "productInfo.jsp"; // 도서 상세 화면으로 이동
    }

    // [도서 등록 페이지 핸들러 메서드] 관리자 전용 도서 정보 추가 인풋 폼 화면 파일명을 반환
    private String addProduct(HttpServletRequest request, HttpServletResponse response) {
        return "addProduct.jsp";
    }

    /**
     * [로그인 처리 프로세스]
     * 사용자가 입력한 ID 정보를 바탕으로 세션을 생성
    */
    private String processLogin(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        request.setCharacterEncoding("UTF-8");
        String id = request.getParameter("id");
        String password = request.getParameter("password"); // 실습 테스트 목적이므로 인증 로직 패스 후 바로 바인딩

        // 서버에 세션(Session) 공간을 요청하고 생성
        HttpSession session = request.getSession();

        // 로그인에 성공한 사용자의 ID를 'sessionId'라는 키값으로 세션에 저장하여, 브라우저가 닫힐 때까지 유지
        session.setAttribute("sessionId", id);

        return "welcomeUser.jsp"; // 로그인 완료 후 회원 전용 환영 페이지로 리다이렉트 경로 반환
    }

    /**
     * [로그아웃 처리 프로세스]
     * 세션을 완전히 파기하여 로그아웃 상태를 만듭니다.
    */
    private String processLogout(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        request.setCharacterEncoding("UTF-8");

```

```

// 현재 브라우저와 매핑된 사용자의 세션 객체를 가져옴
HttpSession session = request.getSession();

// session.invalidate(): 세션 공간에 저장된 모든 정보(로그인 ID 등)를 한 번에 폭파(만료)시킴
session.invalidate();

return "welcome.jsp"; // 로그아웃 후 다시 비회원용 메인 화면으로 이동하도록 지정
}

/**
 * [★도서 등록 처리 프로세스 - 핵심 기능]
 * 업로드된 이미지 파일을 물리 서버에 저장하고, 품 필드 값을 자바 객체에 바인딩하여 DB에 밀어 넣음
 */
private String processAddProduct(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    request.setCharacterEncoding("UTF-8");

    Book b = new Book(); // 품 값을 채워 넣을 빈 도서 데이터 모델(DTO) 객체 생성

    try {
        // 1. 이미지 파일 업로드 처리
        // addProduct.jsp내의 <input type="file" name="file"> 태그로부터 바이너리 파일 데이터 가져옴
        Part part = request.getPart("file");
        // 하단의 getFilename 사용자 정의 메서드를 호출하여 파일의 실제 이름(예: "book_cover.jpg")을 추출함
        String fileName = getFilename(part);

        // 파일 이름이 정상적으로 존재할 경우에만 파일 저장 로직 실행
        if(fileName != null && !fileName.isEmpty()){
            // @MultipartConfig에 정의된 위치("c:/Temp/img") 폴더에 해당 파일 이름으로 실제 디스크 저장
            (Write)을 수행
            part.write(fileName);
        }

        // 2. 아파치 Commons BeanUtils 오픈소스를 활용한 '자동 파라미터 매핑'
        // addProduct.jsp 폼에 입력된 수많은 필드값(productId, name, unitPrice 등)의 name 속성과
        // Book 자바 객체의 Setter 메서드 명을 시스템이 자동으로 매칭하여 b 객체에 데이터를 한 번에 주입
        BeanUtils.populate(b, request.getParameterMap());

        // 3. 자바 객체 내 이미지 경로 수정 보완
        // DB 및 화면 렌더링에 사용할 상대 경로 형식("/img/파일명")으로 조합하여 Book 객체의 이미지 필드에 셋팅
        b.setProductImg("/img/" + fileName);

        // 4. 최종 데이터베이스 저장 실행
        // 완벽하게 값이 셋팅된 Book 객체(b)를 DAO의 insert 메서드로 넘겨 H2 데이터베이스에 최종 Insert 쿼리
        // 를 실행
        .

        dao.insert(b);
    }
}

```

```

    } catch (Exception e) {
        e.printStackTrace();
        ctx.log("도서 추가 과정에서 문제 발생!!"); // 에러 발생 시 톱캣 콘솔에 로그를 남김
    }

    // 저장이 완료되면 페이지 이동 없이 바로 도서 목록 조회 메시지를 연쇄 호출하여 최신 목록 화면으로 새로고침
    return productInfoList(request, response);
}

/**
 * [getFilename 헬퍼 메서드]
 * HTTP 통신 패킷의 Multipart 헤더 정보(Content-Disposition)에서 순수 파일 이름만 잘라내는 유틸리티 공정
 */
private String getFilename(Part part) {
    String fileName = null;

    // part 객체에서 헤더 정보 스트링을 읽어옴
    // 출력 형태 결과 예시 -> form-data; name="file"; filename="java_book.jpg"
    String header = part.getHeader("content-disposition");
    System.out.println("Header => " + header);

    // 헤더 문자열에서 "filename=" 이 시작되는 위치 인덱스를 찾음
    int start = header.indexOf("filename=");

    // "filename=" 문자열 길이(10자) 뒤부터 맨 마지막 쌍따옴표 바로 전까지 슬라이싱하여 파일명만 쏙 빼냄
    fileName = header.substring(start + 10, header.length() - 1);

    ctx.log("추출된 파일명: " + fileName);
    return fileName; // 추출 완료된 순수 파일명 반환
}
}

```

2. Java Model 레이어 소스코드 (Entity DTO & DAO 데이터 접근)

■ Book.java (자바 빈즈 규격의 도서 객체 모델)

```

package ex11;

public class Book {
    // 도서 인스턴스 전용 데이터 필드 정의
    private String productId; // 도서 고유 코드 (Primary Key 연동 식별값)
    private String pname; // 도서 제품 제목
    private Integer unitPrice; // 단가 가격 지표
    private String description; // 도서 상세 설명 줄글
    private String manufacturer; // 출판사 회사명
    private String category; // 카테고리 기획 분류 코드
    private Integer unitsInStock; // 현재 창고에 누적 보관된 재고 수량
}

```

```

private String condition;      // 상품 상태 조건 정보 (신규도서 / 중고도서 / 재생도서)
private String productImg;    // 업로드 저장 폴더 내 맵핑 처리될 가상 파일명 주소

// 아파치 BeanUtils 데이터 리플렉션 가동을 유도하기 위한 기본 생성자
public Book() {}

// 캡슐화 규칙 유지를 위한 각 멤버 변수별 필수 Getter/Setter 정의
public String getProductId() { return productId; }
public void setProductId(String productId) { this.productId = productId; }

public String getPname() { return pname; }
public void setPname(String pname) { this.pname = pname; }

public Integer getUnitPrice() { return unitPrice; }
public void setUnitPrice(Integer unitPrice) { this.unitPrice = unitPrice; }

public String getDescription() { return description; }
public void setDescription(String description) { this.description = description; }

public String getManufacturer() { return manufacturer; }
public void setManufacturer(String manufacturer) { this.manufacturer = manufacturer; }

public String getCategory() { return category; }
public void setCategory(String category) { this.category = category; }

public Integer getUnitsInStock() { return unitsInStock; }
public void setUnitsInStock(Integer unitsInStock) { this.unitsInStock = unitsInStock; }

public String getCondition() { return condition; }
public void setCondition(String condition) { this.condition = condition; }

public String getProductImg() { return productImg; }
public void setProductImg(String productImg) { this.productImg = productImg; }
}

```

■ BookDAO.java (H2 데이터베이스 연동 및 전용 SQL 트랜잭션 수행 객체)

```

package ex11;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class BookDAO {
    // 데이터베이스 커넥션과 SQL 구문 처리를 위한 핵심 인터페이스 변수 선언
    Connection conn = null;
    PreparedStatement pstmt = null;
}

```

```

// 로컬 연동 대상 H2 드라이버 명세 및 내장 메모리 세션 접속 URL 구성 정보
final String JDBC_DRIVER = "org.h2.Driver";
final String JDBC_URL = "jdbc:h2:tcp://localhost/~bookmarketdb";

/**
 * [open 메서드]
 * 지정된 데이터베이스 드라이버를 로드하고 가동 세션 커넥션을 활성화
 */
public void open(){
    try {
        Class.forName(JDBC_DRIVER);
        conn = DriverManager.getConnection(JDBC_URL, "user1", "1234");
    } catch (Exception e) {
        e.printStackTrace();
    }
}

/**
 * [close 메서드]
 * 시스템 누수가 발생하지 않도록 자원이 완료된 JDBC 자원을 선입선출 방식으로 해제 처리
 */
public void close() {
    try {
        if(pstmt != null) pstmt.close();
        if(conn != null) conn.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

/**
 * [insert 메서드]
 * 신규 도서의 정보 및 첨부된 이미지 물리 가상 경로 데이터를 인풋받아 최종 데이터베이스에 삽입
 */
public void insert(Book b) {
    String sql = "INSERT INTO book3(productid, pname, unitprice, description, manufacturer, category, unitsinstock, condition, productimg) values(?,?,?,?,?,?,?,?,?)";
    open(); // 데이터베이스 세션 연결 가동
    try {
        pstmt = conn.prepareStatement(sql);
        // PreparedStatement 인덱스 맵 파라미터에 Book 객체 인스턴스 전용 데이터 바인딩
        pstmt.setString(1, b.getProductid());
        pstmt.setString(2, b.getPname());
        pstmt.setInt(3, b.getUnitPrice());
        pstmt.setString(4, b.getDescription());
        pstmt.setString(5, b.getManufacturer());
        pstmt.setString(6, b.getCategory());
        pstmt.setInt(7, b.getUnitsInStock());
        pstmt.setString(8, b.getCondition());
        pstmt.setString(9, b.getProductImg());
    }
}

```



```

        pstmt.executeUpdate(); // 데이터 저장 쿼리 최종 전송 가동
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        close(); // 안전 자원 반납 처리
    }
}

/**
 * [findAll 메서드]
 * 저장고에 저장된 모든 도서 엔티티를 레코드 배열 단위로 수집 가공하여 반환
 */
public List<Book> findAll() {
    open();
    List<Book> books = new ArrayList<>();
    try {
        pstmt = conn.prepareStatement("select * from book3");
        ResultSet rs = pstmt.executeQuery(); // 결과 데이터 레코드셋 반환
        while(rs.next()) {
            Book b = new Book();
            // DB 테이블 컬럼명과 타겟 자바빈즈 데이터 매핑 주입 공정
            b.setProductId(rs.getString("productid"));
            b.setPname(rs.getString("pname"));
            b.setUnitPrice(rs.getInt("unitprice"));
            b.setDescription(rs.getString("description"));
            b.setManufacturer(rs.getString("manufacturer"));
            b.setCategory(rs.getString("category"));
            b.setUnitsInStock(rs.getInt("unitsinstock"));
            b.setCondition(rs.getString("condition"));
            b.setProductImg(rs.getString("productimg"));
            books.add(b); // 배열 리스트에 주입
        }
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        close();
    }
    return books; // 구성 완료된 전체 리스트 전달
}

/**
 * [find 단건 상세조회 메서드]
 * 도서 아이디 키값을 매개변수로 공급받아 일치하는 행 정보 1행을 객체 형식으로 환원
 */
public Book find(String id) {
    open();
    Book b = new Book();
    try {
        pstmt = conn.prepareStatement("select * from book3 where productid=?");
        pstmt.setString(1, id);
        ResultSet rs = pstmt.executeQuery();
    }
}

```

```

        if(rs.next()) {
            // 단건 데이터 패칭 바인딩 공정
            b.setProductId(rs.getString("productid"));
            b.setPname(rs.getString("pname"));
            b.setUnitPrice(rs.getInt("unitprice"));
            b.setDescription(rs.getString("description"));
            b.setManufacturer(rs.getString("manufacturer"));
            b.setCategory(rs.getString("category"));
            b.setUnitsInStock(rs.getInt("unitsinstock"));
            b.setCondition(rs.getString("condition"));
            b.setProductImg(rs.getString("productimg"));
        }
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        close();
    }
    return b;
}
}

```

3. View (JSP 레이어 Presentation 컴포넌트 소스코드)

■ welcome.jsp (비회원 타겟 홈 메인 뷰)

```

<%@ page contentType="text/html; charset=utf-8"%>
<%@ page import="java.util.Date"%>
<html>
<head>
<!-- 디자인 레이아웃을 위한 외부 부트스트랩 CSS 파일 연결 공정 -->
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/
bootstrap.min.css">
<title>Welcome 환영페이지</title>
</head>
<body>
<!-- 상단 네비게이션 공통 바 컴포넌트 인클루드 동적 결합 -->
<jsp:include page="menu.jsp"/>
<div class="jumbotron">
<div class="container">
</div>
</div>
<h2 class="display-3" style="text-align:center;">북스토어에 오신 것을 환영합니다</h2>
<div class="container">
<div class="text-center">
<h3>Welcome to Book Market!</h3>
<%!
// 전역 성격의 서블릿 멤버 변수 선언문 (방문 카운팅 누적 기록 공간)
int pageCount = 0;
void addCount() {

```

```

        pageCount++;
    }
    %>
<%
// 브라우저 웰컴 페이지 진입 시마다 서블릿 멤버 메서드 addCount 작동 제어
addCount();
%>
<p>
이 사이트 방문자로는 <span style="color:red"> <%=pageCount%> </span> 번째 입니다.
</p>
</div>
<hr>
</div>
<!-- 하단 카피라이트 공통 바 레이아웃 인클루드 모듈 연결 -->
<jsp:include page="footer.jsp" />
</body>
</html>

```

■ welcomeUser.jsp (보안 세션 로그인 연동 회원 메인 뷰)

```

<%@ page contentType="text/html; charset=utf-8"%>
<%@ page import="java.util.Date"%>
<html>
<head>
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/
bootstrap.min.css">
<title>Welcome 사용자 페이지</title>
</head>
<body>
<jsp:include page="menu.jsp"/>
<div class="jumbotron">
<div class="container">
    <h2 class="display-3">
        <!-- 컨트롤러 세션 바인딩 영역의 sessionId 속성값을 EL 표기법 형태로 추출 및 매핑 -->
        <span style="color: blue">${sessionId}</span>님 반갑습니다!!
    </h2>
</div>
</div>
<div class="container">
<div class="text-center">
<h4>Welcome to Book Market!</h4>
<%!
// 회원 전용 접속 로그 카운터용 전역 변수 컴포넌트 선언
int userPageCount = 0;
void addUserCount() {
    userPageCount++;
}
%>
<%
addUserCount();

```

```

%>
<p>
이 사이트 로그인 사용자는 <span style="color:red"><%=userPageCount%></span> 번째입니다.
</p>
</div>
<hr>
</div>
<jsp:include page="footer.jsp" />
</body>
</html>

```

■ menu.jsp (동적 권한 제어형 상단 네비게이션 메뉴 컴포넌트)

```

<%@ page contentType="text/html; charset=utf-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<nav class="navbar navbar-expand navbar-dark bg-dark">
<div class="container">
<div class="navbar-header">
<!-- 컨트롤러 welcome 비즈니스 액션으로 유도하는 홈 링크 설정 -->
<a class="navbar-brand" href="/exercise/bm3control?action=welcome">Home</a>
</div>
<div>
<ul class="navbar-nav mr-auto">
<!-- JSTL choose 분기 제어문을 통해 로그인 세션(sessionId) 존재 유무에 맞춰 상단 버튼을 동적 변경 스위칭 처리 -->
<c:choose>
  <!-- 세션값이 비어있는 비회원 미인증 상태일 때: 로그인 버튼 표출 -->
  <c:when test="${empty sessionId}">
    <li class="nav-item">
      <a class="nav-link" href="<c:url value='/bm3control?action=login'/>">로그인</a>
    </li>
  </c:when>
  <!-- 회원 인증 세션정보가 실시간 유지상태인 경우: 닉네임과 로그아웃 액션 연동 버튼 표출 -->
  <c:otherwise>
    <li style="padding-top: 7px; color: white;">[${sessionId}님] </li>
    <li class="nav-item">
      <a class="nav-link" href="<c:url value='/bm3control?action=processLogout'/>">로그아웃</a>
    </li>
  </c:otherwise>
</c:choose>
<!-- 공통 메뉴 인터페이스 라인 구성 -->
<li class="nav-item"><a class="nav-link" href="<c:url value='/bm3control?action=productInfoList'/>">도서목록</a></li>
<li class="nav-item"><a class="nav-link" href="<c:url value='/bm3control?action=addProduct'/>">도서등록</a></li>
</ul>
</div>
</div>
</nav>

```

■ footer.jsp (공통 바닥 저작권 명세 레고 블록 뷰)

```
<footer class="container">
<hr>
<!-- 시스템 하단 저작권 텍스트 표기 규정 선언 -->
<p>&copy; BookMarket</p>
</footer>
```

■ login.jsp (사용자 로그인 유도 계정 인터페이스 인풋 창)

```
<%@ page contentType="text/html; charset=utf-8"%>
<html>
<head>
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/
bootstrap.min.css">
<title>Login</title>
</head>
<body>
<jsp:include page="./menu.jsp" />
<div class="jumbotron">
<h1 class="display-3" style="text-align:center;">로그인</h1>
</div>
<div class="container" align="center">
<h5 class="form-signin-heading">아이디/패스워드를 입력하세요</h5>
<div class="col-md-4 col-md-offset-4">
<!-- 입력된 계정정보를 서버릿의 processLogin 처리 액션 분기 경로로 전달 가동하는 서브밋 폼 구성 -->
<form class="form-signin" action="/exercise/bm3control?action=processLogin" method="post">
<div class="form-group">
<label for="inputUserName" class="sr-only">User Name</label>
<input type="text" class="form-control" placeholder="ID" name="id" required autofocus>
</div>
<div class="form-group">
<label for="inputPassword" class="sr-only">Password</label>
<input type="password" class="form-control" placeholder="Password" name="password"
required>
</div>
<button class="btn btn-lg btn-success btn-block" type="submit">로그인</button>
</form>
</div>
</div>
<jsp:include page="footer.jsp" />
</body>
</html>
```

■ productInfoList.jsp (전체 도서 카달로그 격자 데이터 배치 뷰)

```
<%@ page contentType="text/html; charset=utf-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<html>
```

```

<head>
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/
bootstrap.min.css">
<title>도서 정보 목록</title>
</head>
<body>
<jsp:include page="menu.jsp" />
<div class="jumbotron">
<div class="container">
<h1 class="display-3">도서목록</h1>
</div>
</div>
<div class="container">
<div class="row" align="center">
<!-- 컨트롤러 레이어에서 request.setAttribute("books", ...) 보관해 준 컬렉션을 순회하는 코어 루프 태그 구동 -->
<c:forEach var="b" varStatus="i" items="${books}">
<div class="col-md-4" style="margin-bottom: 25px;">
  <!-- 이미지 저장 디렉터리에 맵핑된 이미지 소스 주소와 개별 도서 속성값을 EL 식으로 바인딩 처리 -->
  
  <h3>${b.pname}</h3>
  <p>${b.description}</p>
  <p>${b.unitPrice}원</p>
  <p>
    <!-- 상세조회 라우터 명령과 타겟 대상 productId 기본키 식별자 파라미터를 넘겨주도록 하이퍼링크 설계 -->
    <a href="/exercise/bm3control?action=productInfo&id=${b.productId}" class="btn btn-
secondary" role="button">
      상세 정보 &raquo;
    </a>
  </p>
</div>
</c:forEach>
</div>
</div>
<jsp:include page="footer.jsp" />
</body>
</html>

```

■ productInfo.jsp (도서 프로필 명세 정보 단건 레이아웃 뷰)

```

<%@ page contentType="text/html; charset=utf-8"%>
<html>
<head>
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/
bootstrap.min.css">
<title>도서 상세 정보</title>
</head>
<body>
<jsp:include page="menu.jsp" />
<div class="jumbotron">
<div class="container">

```

```

<h1 class="display-3">도서 정보</h1>
</div>
</div>
<div class="container">
<div class="row">
<div class="col-md-5">
    <!-- 컨트롤러에서 request 영역에 'b'라는 별칭으로 전달한 단건 Book 도서 객체 이미지 및 바인딩 데이터 가공 표출 -->
    
</div>
<div class="col-md-7">
    <h3>{b.pname}</h3>
    <p>{b.description}</p>
    <p><b>도서 코드:</b><span class="badge badge-danger">{b.productId}</span></p>
    <p><b>출판사</b>: {b.manufacturer}</p>
    <p><b>분류</b>: {b.category}</p>
    <p><b>재고 수</b>: {b.unitsInStock}</p>
    <p><b>상태</b>: {b.condition}</p>
    <h4 style="color:#c53030;">{b.unitPrice}원</h4>
    <p>
        <a href="#" class="btn btn-info">도서 주문 &raquo;</a>
        <a href="/exercise/bm3control?action=productInfoList" class="btn btn-secondary">도서 목
    </p>
    </div>
</div>
</div>
<jsp:include page="footer.jsp" />
</body>
</html>

```

■ addProduct.jsp (바이너리 멀티파트 전송 기능 탑재 도서 생성 인풋 폼)

```

<%@ page contentType="text/html; charset=utf-8"%>
<html> <
head>
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/
bootstrap.min.css">
<title>도서 등록</title>
</head>
<body>
<jsp:include page="menu.jsp" />
<div class="jumbotron">
<div class="container">
<h1 class="display-3">도서 등록</h1>
</div>
</div>
<div class="container">
<!-- [enctype="multipart/form-data" 필수 주석 설정]
    도서 커버 사진 등 바이너리 파일 스트림 업로드를 제어하기 위해 폼 인코딩 방식을 멀티파트로 필수 격상 변경 지정 -->

```

```

<form name="newProduct" action="/exercise/bm3control?action=processAddProduct"
enctype="multipart/form-data" class="form-horizontal" method="post">
  <div class="form-group row">
    <label class="col-sm-2">도서 코드</label>
    <div class="col-sm-3">
      <input type="text" name="productId" class="form-control">
    </div>
  </div>
  <div class="form-group row">
    <label class="col-sm-2">도서명</label>
    <div class="col-sm-3">
      <!-- 각 input 컴포넌트의 name 속성은 컨트롤러 자바 BeanUtils.populate() 자동 주입 공정의 Setter
명과 직접적으로 일치 작동 -->
      <input type="text" name="pname" class="form-control">
    </div>
  </div>
  <div class="form-group row">
    <label class="col-sm-2">가격</label>
    <div class="col-sm-3">
      <input type="text" name="unitPrice" class="form-control">
    </div>
  </div>
  <div class="form-group row">
    <label class="col-sm-2">상세 정보</label>
    <div class="col-sm-5">
      <textarea name="description" cols="50" rows="2" class="form-control"></textarea>
    </div>
  </div>
  <div class="form-group row">
    <label class="col-sm-2">출판사</label>
    <div class="col-sm-3">
      <input type="text" name="manufacturer" class="form-control">
    </div>
  </div>
  <div class="form-group row">
    <label class="col-sm-2">분류</label>
    <div class="col-sm-3">
      <input type="text" name="category" class="form-control">
    </div>
  </div>
  <div class="form-group row">
    <label class="col-sm-2">재고 수</label>
    <div class="col-sm-3">
      <input type="text" name="unitsInStock" class="form-control">
    </div>
  </div>
  <div class="form-group row">
    <label class="col-sm-2">상태</label>
    <div class="col-sm-5">
      <input type="radio" name="condition" value="신규도서" checked> 신규 도서
      <input type="radio" name="condition" value="중고도서"> 중고 도서
    </div>
  </div>

```



```

        <input type="radio" name="condition" value="재생도서"> 재생 도서
    </div>
</div>
<div class="form-group row">
    <label class="col-sm-2">도서 사진</label>
    <div class="col-sm-3">
        <!-- 컨트롤러의 request.getPart("file") 로직과 매핑 호출되는 파일 업로드 전용 컴포넌트 선언 -->
        <input type="file" name="file" class="form-control">
    </div>
</div>
<div class="form-group row">
    <div class="col-sm-offset-2 col-sm-10">
        <input type="submit" class="btn btn-primary" value="등록">
    </div>
</div>
</form>
</div>
<jsp:include page="footer.jsp" />
</body>
</html>

```

4. 웹 프로젝트 인프라 설정 파일 (Config 레이어)

■ pom.xml (메이븐 의존성 및 컴파일러 규격 빌드 아키텍처 가이드)

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://
maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>exercise</groupId>
    <artifactId>exercise</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>war</packaging>
    <build>
        <resources>
            <resource>
                <directory>src/main/java</directory>
                <excludes>
                    <exclude>**/*.java</exclude>
                </excludes>
            </resource>
        </resources>
        <plugins>
            <plugin>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.8.1</version>
                <configuration>
                    <!-- 자바 실행 릴리즈 환경을 버전 17 규격으로 매핑 강제화 -->
                    <release>17</release>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>

```

```

        </configuration>
    </plugin>
    <plugin>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.2.3</version>
    </plugin>
</plugins>
</build>
<dependencies>
    <!-- JSTL 동적 화면 제어 코어 태그 라이브러리 의존성 주입 -->
    <dependency>
        <groupId>javax.servlet</groupId>
        <artifactId>jstl</artifactId>
        <version>1.2</version>
    </dependency>
    <!-- H2 데이터베이스 드라이버 연동 엔진 라이브러리 추가 -->
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <version>1.4.200</version>
    </dependency>
    <!-- 파라미터를 자바빈즈 프로퍼티 세터에 자동 연동하는 아파치 BeanUtils 모듈 주입 -->
    <dependency>
        <groupId>commons-beanutils</groupId>
        <artifactId>commons-beanutils</artifactId>
        <version>1.9.4</version>
    </dependency>
    <!-- cos 파일 업로드 멀티파트 유틸리티 엔진 탑재 -->
    <dependency>
        <groupId>com.servlets</groupId>
        <artifactId>cos</artifactId>
        <version>05Nov2002</version>
    </dependency>
</dependencies>
</project>

```

■ server.xml (톰캣 내부 물리 리소스 로컬 가상 디렉터리 경로 매핑 레이아웃)

```

<?xml version="1.0" encoding="UTF-8"?>
<!-- 톰캣 서버 엔진의 글로벌 하위 가상 컨텍스트 영역 선언 부 -->
<Server port="8005" shutdown="SHUTDOWN">
    <Service name="Catalina">
        <Engine name="Catalina" defaultHost="localhost">
            <Host name="localhost" appBase="webapps" unpackWARs="true" autoDeploy="true">

                <!-- [★가상 이미지 매핑 디렉터리 경로 핵심 중요 주석]
                디스크 내부 "c:/Temp/img" 실제 물리 저장 공간을 브라우저 가상 매핑 루트 주소 "/img"로 가동 연결하는 설정 -->
                <Context docBase="c:/Temp/img" path="/img" reloadable="true"/>
            </Host>
        </Engine>
    </Service>
</Server>

```

```
<!-- 서블릿 핵심 모듈 실습 프로젝트를 서비스 가동 루트 자원에 동적 할당 처리 -->  
<Context docBase="exercise" path="/exercise" reloadable="true"  
source="org.eclipse.jst.jee.server:exercise"/>  
  
</Host>  
</Engine>  
</Service>  
</Server>
```